

Técnico em Informática Integrado
Programação Orientada a Objetos
Prof. Rafael Henrique Dalegrave Zottesso

Atividade Prática: Desenvolvimento do Jogo "Coletor de Tesouros com Desafios POO"

1. Objetivo Geral

O objetivo desta atividade é aplicar os conceitos fundamentais de Programação Orientada a Objetos (Herança, Sobrescrita de Métodos, Encapsulamento de Estados e Polimorfismo) utilizando a biblioteca Python Arcade. Os alunos deverão construir um jogo interativo 2D com gerenciamento de janelas, movimentação por física vetorial, controle de eventos por teclado, listas otimizadas de elementos visuais, detecção de colisões e renderização de textos informativos.

2. Descrição do Projeto

Os alunos deverão criar um jogo em que um personagem principal (controlado pelo teclado) precisa coletar um conjunto de moedas estáticas e moedas especiais em movimento espalhadas de forma aleatória pela tela. Para adicionar complexidade mecânica e aplicar conceitos avançados de POO, um inimigo dinâmico deve ser implementado. Diferente das moedas, este inimigo nunca é removido do jogo quando colide com o jogador; ele continua vagando persistentemente pelo mapa. Toda vez que o inimigo encostar no player, a contagem de moedas coletadas deve ser diminuída e um texto de alerta de dano deve ser apresentado na tela. Um segundo inimigo também deve existir no mapa, mas esse inimigo é removido ao colidir com o player e deve nascer em uma coordenada aleatória do mapa e começar a se mover a partir dessa nova posição. O jogo também deve exibir a pontuação em tempo real e possuir uma tela de finalização.

3. Visão Geral dos Elementos do Jogo (Sprites)

Elemento (Classe)	Tipo de Herança	Comportamento de Movimentação	Efeito na Colisão com o Player
Player	arcade.Sprite	Controlado por setas ou WASD; travado nas bordas. Inverte a textura horizontalmente conforme a direção.	Elemento centralizador das colisões.

Elemento (Classe)	Tipo de Herança	Comportamento de Movimentação	Efeito na Colisão com o Player
Moeda	arcade.Sprite	Estática (velocidade zerada). Distribuída aleatoriamente no início da partida.	Incrementa +1 ponto e é destruída (removida da lista de sprites).
MoedaEspecial	arcade.Sprite	Movimentação contínua com efeito rebote (inversão de vetor) ao tocar nas bordas da tela.	Incrementa +5 pontos e é destruída (removida da lista de sprites).
Inimigo	arcade.Sprite	Movimentação contínua e autônoma pelo mapa (pode utilizar padrão de rebote ou direção própria).	NÃO deve ser removido. Permanece ativo na tela, diminui a contagem de moedas e dispara um texto de alerta.
InimigoEspecial	arcade.Sprite	Movimentação contínua e autônoma pelo mapa (pode utilizar padrão de rebote ou direção própria).	Diminui -1 ponto da pontuação. É removido quando colide e nasce em uma posição aleatória do mapa, começando a se mover novamente.

4. Requisitos Técnicos Detalhados

A. Estrutura de Janela e Ciclo de Vida (arcade.Window)

- **Configuração Inicial:** Criar uma subclasse de arcade.Window chamada JanelaJogo. A janela deve possuir as dimensões de 800x600 pixels, título personalizado e uma cor de fundo definida via constantes da biblioteca (como arcade.color.AMAZON) ou tuplas RGB no método construtor (`__init__`).
- **Gerenciamento de Métodos:** Sobrescrever obrigatoriamente os métodos `on_draw()` (garantindo a limpeza prévia do frame com `self.clear()`) e `on_update(self, delta_time)` para a lógica contínua de atualização.

B. Classes de Sprites e Herança (arcade.Sprite)

- **Classe Player:** Deve herdar de arcade.Sprite. No construtor, deve carregar uma imagem padrão com escala apropriada. Deve armazenar as texturas para movimento

à direita e à esquerda. No método `update()`, a propriedade `self.texture` deve mudar dinamicamente conforme a direção horizontal do movimento (`change_x`).

- **Classe Moeda (Estática):** Deve herdar de `arcade.Sprite`. Deve ser posicionada de forma estática e não possuir velocidade inicial no mapa.
- **Classe MoedaEspecial (Dinâmica):** Deve herdar de `arcade.Sprite`. Deve mover-se de forma contínua e implementar obrigatoriamente o efeito rebote (multiplicar `change_x` ou `change_y` por -1) ao colidir com as bordas da tela.
- **Classe Inimigo (Polimorfismo e Persistência):** Deve herdar de `arcade.Sprite`. O inimigo deve ser programado para se mover pelo mapa de forma independente. Diferente dos itens coletáveis, ele **nunca deve ser removido das listas de sprites** após colidir com o jogador. Ele deve continuar sua trajetória, vagando pelo mapa e permitindo múltiplas colisões ao longo do tempo.
- **Classe InimigoEspecial:** Deve herdar de `arcade.Sprite`. Possui movimentação contínua e autônoma pelo mapa. Ao colidir com o jogador, o inimigo deve ser removido e imediatamente recriado em uma coordenada aleatória do mapa, reiniciando sua movimentação a partir dessa nova posição.

C. Física, Limites de Tela e Inputs do Teclado

- **Física de Movimentação:** Utilizar os atributos `change_x` e `change_y` nas classes dos sprites para atualizar as posições `center_x` e `center_y` a cada frame dentro do método `update()`.
- **Contenção de Bordas do Player:** O Player não pode sair dos limites visíveis da janela. Deve-se usar as propriedades `left`, `right`, `top` e `bottom` do sprite para travá-lo nas extremidades internas (0 a 800 no eixo X; 0 a 600 no eixo Y).
- **Eventos de Entrada:** Sobrescrever `on_key_press()` e `on_key_release()`. As setas direcionais ou teclas WASD devem alterar a velocidade do jogador. Ao soltar a tecla, a velocidade correspondente deve ser zerada. A tecla ESCAPE deve fechar a janela imediatamente via `arcade.close_window()`.

D. Otimização, Coleta e Regras de Negócio (`arcade.SpriteList`)

- **Listas de Desenho:** Todos os elementos devem ser armazenados e renderizados por meio de objetos `arcade.SpriteList` para garantir a otimização de hardware.
- **Loop de Geração:** Gerar um grupo de pelo menos 25 moedas distribuídas aleatoriamente na janela utilizando estruturas de repetição (`for i in range(...)`) e a biblioteca `random` (`random.randint`).
- **Cálculo de Colisão e Penalidade:** No método `on_update()` da janela, utilizar `arcade.check_for_collision_with_list()` para detectar colisões. Se houver colisão com moedas, elas devem ser removidas com `remove_from_sprite_lists()`. Se houver colisão com o inimigo, a pontuação de moedas deve ser decrementada.
- **Interface Textual (HUD):** Exibir o texto com a pontuação atualizada no topo da tela utilizando `arcade.draw_text()`. Adicionalmente, quando o jogador colidir com o inimigo, um texto explicativo de alerta (ex: "Cuidado! Você foi atingido!") deve ser desenhado

na tela para notificar o usuário instantaneamente sobre a perda de pontos.

5. Itens Obrigatórios de Implementação (Checklist para o Aluno)

1. Código totalmente escrito em Python com uso explícito de Type Hints em argumentos e retornos.
2. Arquivo contendo a validação de execução principal (if `__name__ == "__main__":` `main()`).
3. Classe `JanelaJogo` herdando de `arcade.Window` com chamadas corretas de `super().__init__()`.
4. Classe `Player` com inversão de textura dependendo do sentido do movimento.
5. Geração automatizada e aleatória das moedas estáticas em uma `SpriteList`.
6. Implementação de uma moeda móvel rápida com cálculo exato de inversão vetorial (rebote) nas paredes.
7. Classe `Inimigo` configurada para nunca sumir da tela após o toque, mantendo o movimento contínuo pelo cenário.
8. Implementação da classe `InimigoEspecial` com lógica de colisão, remoção e reposicionamento aleatório no mapa.
9. Mecânica de redução de pontos e exibição de texto de aviso de colisão com o inimigo na interface gráfica.
10. Métodos de teclado configurados para evitar que o player se desloque infinitamente após soltar o botão.
11. HUD funcional desenhando os pontos atualizados de forma limpa na tela.